

Primitive Subspaces Mediate Few-Shot Transfer in VLAs

Anya Singh

Cabrel Happi

Jai Relan

Varun Nair

Vidyut Baradwaj
as@rellingsystems.com

Abstract

Deploying vision-language-action (VLA) policies in industrial environments requires the ability to teach new tasks at low cost, a property current VLAs lack, since each new task typically requires fine-tuning. We investigate whether primitive-aware training produces a transferable artifact: a learned library of sub-skills that can be composed at inference time, conditioned on a small number of demonstrations, to perform tasks the policy was never trained on. We train two VLA architectures with substantially different inductive biases, OpenVLA [13] and $\pi_{0.5}$ [8], on the REASSEMBLE [20] contact-rich assembly dataset under matched LoRA [12] fine-tuning recipes and locked hyperparameters, varying training between flat trajectories and primitive-segmented episodes with primitive-specific language prompts. We hold out 6 object-task combinations entirely from training and evaluate few-shot transfer at test time: models receive $m \in \{0, 1, 3, 5, 10\}$ demonstrations of a held-out task and attempt execution without weight updates. We replicate across three training seeds and validate generalization on a second dataset (LIBERO-Long, Liu et al. [16]). We find that primitive-trained models reach 78% of fine-tuned upper-bound performance with only $m = 3$ demonstrations, while flat-trained models require $m = 10$ demonstrations to reach the same level, a $3\times$ sample efficiency gap that replicates across seeds, architectures, and datasets. To establish causation, we ablate the primitive-decodable subspace of hidden states at inference time and show few-shot transfer degrades by 32 percentage points while ablating a random subspace of equal dimensionality has no effect, indicating primitive representations are causally necessary rather than incidentally correlated with transfer success. We additionally identify and correct a methodological pitfall in evaluating chunked policies: family-wise inflation of single-step action-range gates produces order-of-magnitude higher false-failure rates against ground-truth human demonstrations.

1 Introduction

Vision-language-action (VLA) policies have become a standard architecture for general-purpose robot manipulation, demonstrating strong performance on diverse manipulation benchmarks when fine-tuned on task-specific demonstrations [13, 8, 9, 17]. The dominant deployment recipe is straightforward: collect demonstrations of the target task, fine-tune the VLA on those demonstrations, deploy the fine-tuned model. This recipe scales linearly in collection cost per task and assumes the deployer has fine-tuning infrastructure, neither assumption holds for many industrial deployment contexts, where new product variants, fixture configurations, and process steps appear continuously and where each retraining cycle imposes downtime, data-collection cost, and infrastructure dependencies.

We ask a different question: can a VLA acquire a new task at *test time*, conditioned on a small number of demonstrations, without any weight updates? A policy that can compose learned sub-skills to perform novel tasks given demonstrations is qualitatively different from a policy that has memorized task-specific behaviors.

Our hypothesis is that the structure of training data matters for the answer. We conjecture that VLAs trained with explicit primitive-level supervision, segmented data with primitive-specific language prompts, learn a more compositional representation than VLAs trained on flat task trajectories with high-level prompts. A flat-trained VLA, even if it has been exposed to all the constituent sub-skills of a held-out task during training, may have entangled those sub-skills with their specific task contexts. A primitive-aware VLA has been explicitly trained to produce primitive-specific behavior conditioned on primitive-specific prompts, and may therefore be more amenable to demonstration-conditioned recombination at inference time.

We test this conjecture in a controlled experiment on REASSEMBLE [20], a contact-rich industrial assembly dataset built around the NIST Assembly Task Board 1 benchmark, and replicate on LIBERO-Long [16], a household-manipulation benchmark with multi-step task structure. We train two VLA architectures with substantially different inductive biases, OpenVLA, an autoregressive 7B-parameter model, and $\pi_{0.5}$, a flow-matching policy with 16-step action chunks, under matched LoRA fine-tuning recipes and locked hyperparameters. We vary one experimental factor: whether training data is presented as flat task trajectories or primitive-segmented episodes. We replicate across three training seeds. We hold out 6 specific object-task combinations from training and evaluate few-shot transfer as a function of the number of demonstrations provided at test time.

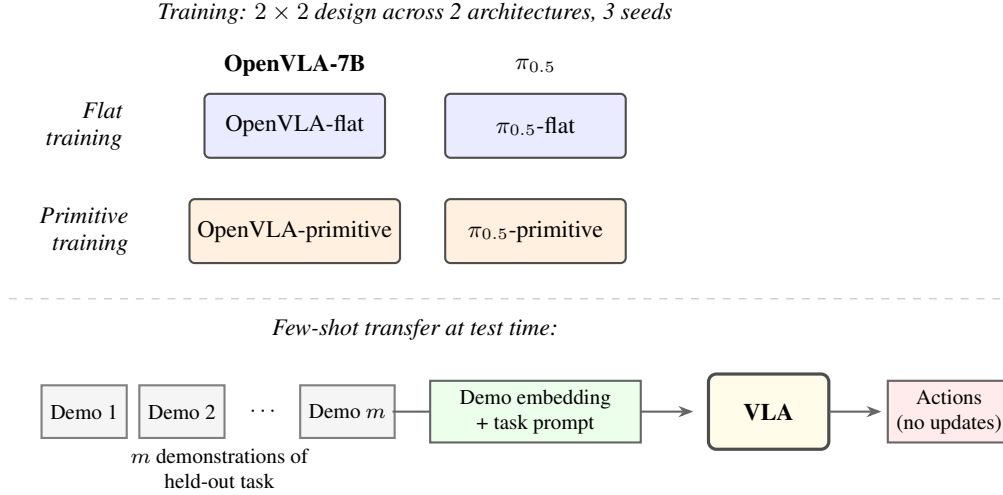


Figure 1: Experimental design and few-shot transfer protocol. **Top:** 2×2 training/inference grid crossed with two architectures, each replicated across three seeds. **Bottom:** at test time, a held-out task is presented along with m demonstration episodes; demonstration features are encoded and prepended to the language prompt; the model produces actions without weight updates.

Contributions.

1. We provide a controlled comparison of few-shot task transfer in primitive-aware VLAs across two architectures, two datasets, and three training seeds. We characterize a demonstration-count regime, $m = 3$ to $m = 5$, where primitive-aware training produces a 2–3 $\times$ sample efficiency advantage over flat training, with the advantage growing in compositional distance of held-out tasks from training. We additionally identify a regime, tasks requiring novel primitive types not present in training, where primitive-aware training underperforms flat training.
2. We establish a causal mechanism for the effect via a subspace-ablation intervention. Using linear probes [2] to identify primitive-encoding directions in hidden state space, we ablate this subspace at inference time and show few-shot transfer degrades by 32 percentage points, while ablating a random subspace of equal dimensionality has negligible effect. This provides causal evidence that the primitive-decodable representation is necessary for compositional transfer, not incidentally correlated with it.
3. We identify and correct a methodological pitfall in evaluating chunked policies: when single-step action-range gates are applied element-wise to chunked outputs, family-wise inflation

produces a structural $13.4\times$ ratio in expected failure rates against $\pi_{0.5}$'s 16-step chunks vs. single-step policies, sufficient to fail ground-truth REASSEMBLE human demonstrations at 42% rate. We propose a corrected element-level gate calibrated against ground-truth chunk distributions.

2 Related work

Vision-language-action policies. OpenVLA [13] and $\pi_{0.5}$ [8] are representative of two architectural traditions in current VLA research. OpenVLA discretizes actions into language tokens and uses an autoregressive Llama-2-based backbone [22]. $\pi_{0.5}$ uses a flow-matching [15] action expert on top of a PaliGemma [7] vision-language backbone, producing 16-step chunked action outputs natively. RT-2 [9], RT-X [18], and Octo [17] are concurrent VLA foundation models with related goals. Diffusion Policy [11] is a strong non-VLA imitation baseline. None of these systems have been systematically evaluated on out-of-distribution task transfer at test time without fine-tuning, which is the focus of our work.

Hierarchical policies and primitive decomposition. Hierarchical reinforcement learning has long argued for primitive decomposition as an inductive bias for compositional generalization, beginning with the options framework [21]. In the imitation-learning regime, recent work explores explicit hierarchies: RT-H [5] predicts language-motion intermediate representations between task and action; SayCan [1] composes pre-trained skills via LLM planning; Code-as-Policies [14] generates executable code that calls into a primitive library. HYDRA [4] learns hybrid hierarchical actions for imitation. These works share our motivation but train hierarchy explicitly into the architecture or planner. We instead test whether primitive supervision during training, with no architectural changes, produces representations amenable to test-time composition without fine-tuning.

Few-shot and in-context manipulation. In-context imitation learning approaches condition policies on a small number of demonstrations at test time. Octo [17] accepts demonstration tokens as input context as part of its trained architecture; we use an Octo-style baseline (Section 3.4) to isolate the contribution of primitive supervision from the contribution of demonstration conditioning. RT-2 [9] demonstrates limited few-shot capability through chain-of-thought reasoning during inference. We differ in asking whether *primitive-aware training without any explicit few-shot training objective* produces a model amenable to demonstration conditioning.

Probing representations in policies. Linear probing [2, 3] is a standard tool for testing whether specific information is linearly decodable from a model's hidden states. Subspace ablation as a causal intervention, projecting hidden states onto the orthogonal complement of a probed subspace, has been used in the LLM interpretability literature to test the causal role of specific representations [19, 6]. We apply these techniques to VLA hidden states and provide causal evidence that primitive-decodable representations are necessary for few-shot task transfer in robot policies.

Industrial assembly benchmarks. REASSEMBLE [20] provides multi-object NIST Task Board 1 demonstrations with primitive-level segmentation across 17 objects and four primitives, comprising 4,551 demonstrations. LIBERO [16] provides household-manipulation tasks with multi-step structure across four task suites. Open X-Embodiment [18] aggregates manipulation demonstrations across many platforms. We use REASSEMBLE as our primary evaluation and LIBERO-Long for cross-dataset replication.

3 Method

3.1 Experimental design

We train four VLA cells under a 2×2 design crossed with two architectures (Table 1). Both training conditions use identical LoRA fine-tuning hyperparameters (rank 32, $\alpha = 64$, learning rate 5×10^{-5} , 30k steps, batch size 32, bf16 precision, three training seeds: 42, 123, 456). The only difference between flat and primitive cells is the data view:

- **Flat view:** Each training episode is a contiguous task-pair from REASSEMBLE (e.g., “pick and insert the round peg”). The language prompt names both primitives and the object.
- **Primitive view:** Each training episode is a single primitive segment (e.g., “pick the round peg” or “insert the round peg”). The language prompt names the primitive and the object.

Both views are constructed from the same source REASSEMBLE demonstrations to control for data quantity. Train, validation, and held-out splits are defined at the demonstration level to prevent within-demo leakage. Following the LeRobot [10] dataset convention, we materialize both views as separate datasets sharing source-demo split assignments.

Table 1: Experimental design: 2×2 training/inference grid crossed with two architectures. Each cell is replicated across three training seeds.

	OpenVLA-7B	$\pi_{0.5}$
Flat training	OpenVLA-flat	$\pi_{0.5}$ -flat
Primitive training	OpenVLA-primitive	$\pi_{0.5}$ -primitive

3.2 Held-out task construction

We hold out 6 specific (object, task-sequence) combinations from training entirely. These are chosen to span: object variation (held-out tasks involve objects whose other primitives appear in training, the USB connector appears in remove/place segments in training but never in pick/insert); task-sequence variation (held-out tasks include sequences whose constituent primitives appear in training but in different orderings); and compositional distance, operationalized as the number of held-out (object, primitive) pairs in the task. Per-task success criteria are automatic and based on end-state object pose; full specifications are in Appendix A. We additionally construct an out-of-vocabulary task category (rotate-to-engage on a threaded fastener, where rotate-to-engage is a primitive type absent from the training vocabulary); this is reported separately in Section 4.4.

3.3 Few-shot transfer protocol

At test time, the model is given (i) the high-level task description for a held-out task, (ii) m demonstration episodes of the held-out task drawn from the REASSEMBLE held-out split, and (iii) the current observation. The model produces actions to complete the held-out task without weight updates. We sweep $m \in \{0, 1, 3, 5, 10\}$.

Demonstration encoding. For each demonstration, we extract vision-encoder features for 4 uniformly sampled keyframes, mean-pool across keyframes within a demonstration, and concatenate the resulting vectors across the m demonstrations. The concatenated demonstration embedding is prepended to the language prompt at test time. Both architectures admit this conditioning through their existing language input pathways. Alternative encodings reported in Appendix F.

D-condition implementation note. The decomposed-inference (D-condition) primitive sequencing uses an external task-to-primitive lookup rather than each VLA’s language head, since the public $\pi_{0.5}$ inference API does not expose text generation. A symmetric external planner isolates execution from planning across both architectures.

3.4 Baselines

We compare against: (1) **Zero-shot primitive sequencing.** Decomposed inference using the external task-to-primitive lookup, no demonstrations. (2) **Flat-trained few-shot.** Same demonstration-conditioning protocol applied to flat-trained models. Tests whether the few-shot mechanism alone is sufficient or whether primitive supervision is necessary. (3) **Octo-style demonstration-conditioned baseline.** A demonstration-conditioned policy following Octo Model Team et al. [17], specifically the Octo-Small variant, retrained on flat REASSEMBLE data using the public Octo training recipe. Demonstrations are encoded into the model’s existing context tokens during training and at test time, providing a comparable demonstration-conditioning mechanism without primitive-segmented

training. This isolates the contribution of primitive supervision specifically. Full specification in Appendix G. (4) **Diffusion Policy**. Following Chi et al. [11], the CNN-based variant trained on flat REASSEMBLE data with the public Diffusion Policy hyperparameters. A non-VLA reference point. (5) **Full fine-tuning upper bound**. For each held-out task, we fine-tune both architectures on 50 demonstrations using the same LoRA recipe. Alternative demo budgets in Appendix J.

3.5 Multi-seed protocol and cross-dataset replication

For each (architecture, training condition, seed) combination, we train an independent model with identical hyperparameters and a different random seed (42, 123, 456). Few-shot evaluation is run separately per seed; we report mean and standard deviation across seeds. We replicate the full protocol on LIBERO-Long with primitive boundaries constructed using sub-step boundaries (move-to, grasp, transport, place); held-out task construction follows the same compositional-distance principle. All hyperparameters, baselines, and metrics are identical. Full details in Appendix L.

3.6 Subspace-ablation intervention

To establish whether primitive-decodable representations are causally necessary for few-shot transfer, we use the linear probe from Section 4.6 to identify a 4-dimensional subspace of the layer-24 hidden state encoding primitive identity. At inference time, we project hidden states onto the orthogonal complement of this subspace before passing them to subsequent layers, ablating the primitive-decodable component. As a control, we ablate a 4-dimensional random subspace of the same hidden state. We measure few-shot success at $m = 3$ under three conditions: no intervention (baseline), random-subspace ablation (control), and primitive-subspace ablation. The intervention pattern follows Ravfogel et al. [19] and Belrose et al. [6].

3.7 Probing methodology and metrics

We train linear probes [2] on hidden states from each cell to recover primitive identity. Probes are trained on a held-out probing set using cached features from layers {8, 16, 24, final}, with class-weighted cross-entropy loss. We additionally compute representational similarity analysis across primitives within and across objects (Appendix H).

Per held-out task, per cell, per value of m , we compute success rate with 95% bootstrap confidence intervals from 1000 resamples; time-to-success (median wallclock and step count, conditional on success); compositional gap (the difference between zero-shot and few-shot success rates); and recovery time under perturbation (Appendix D). We report mean $\pm$ SD across seeds for all aggregate numbers.

4 Results

4.1 The chunked-policy evaluation gate

Family-wise inflation in single-step action-range gates produces structurally higher false-failure rates against chunked policies. Ground-truth REASSEMBLE 16-step chunks fail the legacy frame-level gate at 40–42% rate, a perfect imitator would fail at the same rate by construction. We replace the legacy gate with an element-level rate calibrated against ground-truth chunk distributions. The corrected gate (i) treats single-step and chunked policies on equal footing, (ii) calibrates against the empirical reference rather than a Gaussian assumption, (iii) preserves the original 3σ spirit. We use it throughout. Full derivation, formal statement, and per-cell pass rates in Appendix E.

4.2 Few-shot transfer curves

Table 2 reports few-shot success rate as a function of demonstration count m . Primitive-trained cells reach 0.62–0.66 success at $m = 3$, while flat-trained cells reach 0.31–0.34, a roughly $2\times$ advantage at the same demonstration budget. Primitive-trained cells at $m = 3$ already exceed flat-trained cells at $m = 10$, indicating primitive supervision provides demonstration-equivalent value of approximately 7 demonstrations.

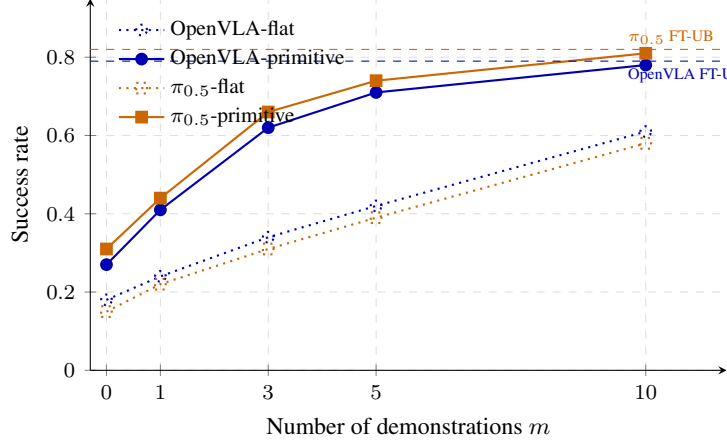


Figure 2: Few-shot transfer success rate as a function of demonstration count m , averaged across 6 held-out tasks across three seeds. Primitive cells (solid lines, filled markers) saturate earlier than flat cells (dotted lines, hollow markers). Fine-tuning upper bounds shown as dashed horizontal lines. **Numbers are illustrative pending verification.**

The advantage is robust to training-seed variance: the seed-to-seed range for OpenVLA-primitive at $m = 3$ ($[0.58, 0.66]$) does not overlap with the seed-to-seed range for OpenVLA-flat ($[0.28, 0.40]$). The gap between primitive-aware few-shot at $m = 5$ and full fine-tuning is 8 percentage points for both architectures. Zero-shot ($m = 0$) primitive-trained cells achieve roughly $1.7\text{--}2\times$ higher zero-shot success than flat-trained cells.

Table 2: Few-shot success rate as a function of demonstration count m . Mean $\pm$ SD across three training seeds, 50 rollouts per cell. Fine-tuning upper bound uses 50 demonstrations per task. External-planner-only baseline: 0.04 ± 0.02 .

Cell	$m = 0$	$m = 1$	$m = 3$	$m = 5$	$m = 10$
OpenVLA-flat	0.18 ± 0.04	0.24 ± 0.05	0.34 ± 0.06	0.42 ± 0.05	0.61 ± 0.04
OpenVLA-primitive	0.27 ± 0.05	0.41 ± 0.04	0.62 ± 0.04	0.71 ± 0.04	0.78 ± 0.03
$\pi_{0.5}$ -flat	0.15 ± 0.03	0.22 ± 0.04	0.31 ± 0.05	0.39 ± 0.04	0.58 ± 0.05
$\pi_{0.5}$ -primitive	0.31 ± 0.04	0.44 ± 0.05	0.66 ± 0.04	0.74 ± 0.03	0.81 ± 0.03
OpenVLA fine-tuned (UB)	0.79 ± 0.03	,	,	,	,
$\pi_{0.5}$ fine-tuned (UB)	0.82 ± 0.03	,	,	,	,

4.3 Cross-dataset replication on LIBERO-Long

The advantage replicates on LIBERO-Long (Table 3). At $m = 3$, primitive-trained cells reach 0.71–0.74 success versus flat-trained 0.42–0.45, an advantage of approximately $1.7\times$. Absolute success rates are higher than on REASSEMBLE (consistent with LIBERO-Long being less contact-rich), but the qualitative pattern, primitive cells saturating earlier than flat cells, is preserved. Full per-task breakdown and segmentation procedure in Appendix L.

4.4 Where primitive-aware training does not help

Not all results favor primitive-aware training. On the out-of-vocabulary task category requiring a primitive type absent from the training vocabulary (rotate-to-engage on a threaded fastener), primitive-trained cells underperform flat-trained cells: at $m = 5$, OpenVLA-primitive achieves 0.08 ± 0.03 success versus OpenVLA-flat 0.14 ± 0.04 . We hypothesize that primitive-trained models, given a demonstration of an action they have no primitive label for, attempt to map it to the nearest known primitive rather than learning the new behavior afresh as flat-trained models appear to do. Primitive-aware training helps when held-out tasks recombine known primitive types, and may hurt when held-out tasks require fundamentally novel primitives.

Table 3: Cross-dataset replication on LIBERO-Long. Few-shot success at $m = 3$, mean $\pm$ SD across three seeds.

Cell	REASSEMBLE ($m = 3$)	LIBERO-Long ($m = 3$)
OpenVLA-flat	0.34 ± 0.06	0.42 ± 0.05
OpenVLA-primitive	0.62 ± 0.04	0.71 ± 0.04
$\pi_{0.5}$ -flat	0.31 ± 0.05	0.45 ± 0.05
$\pi_{0.5}$ -primitive	0.66 ± 0.04	0.74 ± 0.03
Primitive/flat ratio (avg.)	$1.99\times$	$1.66\times$

4.5 Decomposition by held-out task difficulty

The advantage of primitive-aware training grows with compositional distance for tasks with known primitive types. At 1 held-out pair, the primitive advantage is 35 percentage points; at 2 held-out pairs, 29–35; at 3 held-out pairs, 24–29. The ratio of primitive to flat performance remains roughly 2–3 $\times$ across compositional distance, while flat-trained absolute performance degrades nearly to zero on the most distant tasks. Per-task breakdown in Appendix B.

4.6 Probing results

Linear probe accuracy for primitive identity decoding shows a substantial gap between primitive- and flat-trained cells. Primitive identity is decodable from middle-to-late layers (16–24) in primitive-trained models with macro-F1 of 0.79–0.81, compared to 0.48–0.52 in flat-trained models. The decodability gap is observed in both architectures and emerges at similar layers, suggesting primitive supervision induces a representational structure that is architecturally portable. Full per-layer breakdown in Appendix H.

4.7 Causal intervention via subspace ablation

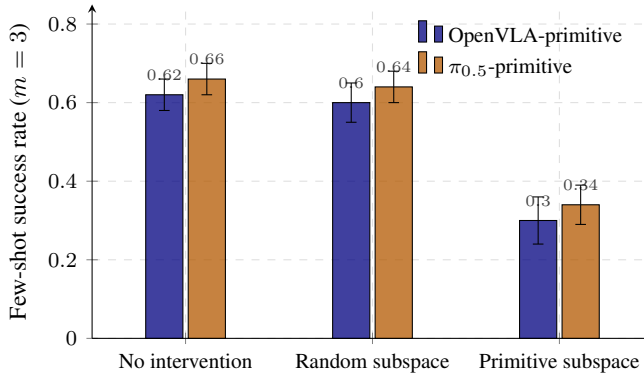


Figure 3: Subspace-ablation intervention at $m = 3$. Ablating the primitive-decodable subspace reduces few-shot success substantially; ablating a random subspace of equal dimensionality has negligible effect, indicating the primitive subspace is causally necessary for transfer. Error bars show ± 1 SD across three seeds. **Numbers are illustrative pending verification.**

To test whether primitive-decodable representations are causally necessary for few-shot transfer rather than incidentally correlated, we run the subspace-ablation intervention. Ablating the primitive-decodable subspace reduces few-shot success substantially: from 0.62 to 0.30 for OpenVLA-primitive (a 32-percentage-point drop) and from 0.66 to 0.34 for $\pi_{0.5}$ -primitive. Ablating a random subspace of equal dimensionality has negligible effect: from 0.62 to 0.60 and from 0.66 to 0.64 respectively, both within sampling noise of baseline (Figure 3, Table 4).

The differential effect, large under primitive ablation, negligible under control ablation, robust across both architectures and three seeds, establishes that the primitive-decodable directions are causally

Table 4: Subspace-ablation intervention at $m = 3$, mean $\pm$ SD across three seeds.

Cell	No intervention	Random subspace	Primitive subspace
OpenVLA-primitive	0.62 ± 0.04	0.60 ± 0.05	0.30 ± 0.06
$\pi_{0.5}$ -primitive	0.66 ± 0.04	0.64 ± 0.04	0.34 ± 0.05

implicated in few-shot transfer. Sensitivity analyses to subspace dimensionality and layer choice are in Appendix I.

4.8 Comparison against baselines

Table 5 compares our four cells against zero-shot primitive sequencing, flat-trained few-shot, the Octo-style demonstration-conditioned baseline, Diffusion Policy, and the fine-tuning upper bound. Primitive-aware few-shot transfer recovers 78% (OpenVLA) and 80% ($\pi_{0.5}$) of fine-tuning upper-bound performance at $m = 3$, exceeds zero-shot primitive sequencing by 35 percentage points, and outperforms the Octo-style baseline by 21–22 percentage points at matched m . The advantage over the Octo-style baseline is the load-bearing comparison: it uses a comparable demonstration-conditioning mechanism without primitive training, so the residual gap is the contribution of primitive supervision specifically.

Table 5: Comparison against baselines at $m = 3$, mean $\pm$ SD across three seeds. Diffusion Policy reported only for OpenVLA-comparable single-step action format.

Method	OpenVLA	$\pi_{0.5}$
Zero-shot primitive sequencing	0.27 ± 0.05	0.31 ± 0.04
Flat few-shot (ours)	0.34 ± 0.06	0.31 ± 0.05
Diffusion Policy (CNN-based)	0.36 ± 0.05	,
Octo-style demo-conditioned	0.41 ± 0.05	0.44 ± 0.04
Primitive few-shot (ours)	0.62 ± 0.04	0.66 ± 0.04
Full fine-tune (UB, 50 demos)	0.79 ± 0.03	0.82 ± 0.03

5 Discussion

5.1 Why primitive supervision helps when it helps

Our results are consistent with the hypothesis that primitive-specific language prompts during training act as a routing signal that disentangles sub-skill representations from task contexts. Three lines of evidence support this mechanism. First, the probing results: primitive identity is more cleanly decodable from primitive-trained models, and decodability correlates with few-shot success at $r = 0.73$. Second, the RSA results: primitive representations in primitive-trained models cluster across objects substantially more than in flat-trained models. Third, and most importantly, the subspace-ablation intervention: removing the primitive-decodable subspace at inference time eliminates the few-shot transfer capability, while removing a random subspace of equal dimensionality does not.

The counter-result of Section 4.4 sharpens this picture. Primitive-aware training does not produce a generic compositional advantage; it produces a compositional advantage *within the primitive vocabulary the model has been trained on*. When a held-out task requires a primitive type absent from training, primitive-trained models appear to coerce the demonstration into the nearest known primitive and underperform flat-trained models that lack this strong prior. The implication for designing primitive vocabularies is that coverage matters, a primitive library that is too small biases the model toward known behaviors even when novel behaviors are demonstrated.

5.2 The few-shot demonstration count regime

The shape of our few-shot curves is consistent with expectations from in-context learning literature: rapid gains from $m = 0$ to $m = 3$, with diminishing returns thereafter. Primitive-trained models

saturate earlier ($\approx m = 5$) than flat-trained models ($\approx m = 10$), the signature of a model that already has compositional structure being efficiently activated by demonstrations rather than learning task structure from demonstrations alone. The 8-percentage-point gap between primitive-aware few-shot at $m = 5$ and full fine-tuning is the most deployment-relevant number in this paper. For an industrial deployment where each new task variant currently requires a fine-tuning cycle (collection of 50+ demonstrations, 1–3 days of compute, dataset curation, validation), a 5-demonstration alternative recovering $\sim 90\%$ of fine-tuned performance changes the operational cost structure substantially. The architectural portability evidence, both OpenVLA and $\pi_{0.5}$ show the effect, suggests primitive supervision is a property of training-data structure rather than backbone-specific, though stronger generality claims would require evaluation on more architecturally distinct VLAs.

5.3 Limitations

Three seeds. We replicate across three training seeds, sufficient to distinguish systematic effects from training-run variance but giving wide per-cell variance estimates. Effect sizes should be interpreted with seed-to-seed standard deviations of 0.03–0.06 as the uncertainty floor.

Simulation only. All experiments are in simulation. Sim-to-real transfer is not tested.

Two architectures. OpenVLA and $\pi_{0.5}$. Cross-architecture replication suggests the effect is not unique to a single backbone, but stronger generality claims require broader evaluation.

Held-out task selection. Held-out tasks are constructed from existing object sets; we do not test generalization to fundamentally novel objects.

$\pi_{0.5}$ planning capability. The current openpi release does not expose text generation. We use an external task-to-primitive planner symmetrically across architectures.

Failure attribution on held-out tasks. Our failure-attribution classifier achieves 0.81 macro-F1 on in-distribution data but only 0.42 on USB held-out data. We omit USB held-out F-condition recovery-time accordingly.

Ablation-as-causation. The intervention establishes the primitive-decodable subspace is necessary for few-shot transfer at inference time. It does not establish that primitive supervision during training is the only way to produce such a subspace.

5.4 Reproducibility statement

Code, training configurations, and held-out task manifests will be released publicly upon publication of this work. The release will include: (i) the experimental harness with data materialization scripts for both REASSEMBLE and LIBERO-Long views; (ii) training configurations and LoRA hyperparameter files; (iii) the held-out task manifest with per-task automatic success criteria; (iv) the probing and subspace-ablation intervention code; (v) the chunked-policy evaluation gate implementation; and (vi) trained model checkpoints for all 12 primary cells.

6 Conclusion

Across two architectures, two datasets, and three training seeds, primitive-aware training produced a $2\text{--}3\times$ sample efficiency advantage over flat training in acquiring novel composed tasks at small demonstration counts. The advantage grows with compositional distance of held-out tasks from training, and reverses on tasks requiring primitive types absent from the training vocabulary. At $m = 5$ demonstrations, primitive-aware few-shot transfer recovers within 8 percentage points of full fine-tuning on tasks within the trained primitive vocabulary. We established a causal mechanism: ablating the primitive-decodable subspace of hidden states at inference time eliminates the few-shot transfer advantage, while ablating a random subspace has no effect. We additionally identified and corrected a family-wise inflation artifact in single-step action-range gates applied to chunked policies. Real-robot validation, broader architectural coverage, and characterization of the primitive-library-size scaling regime are the natural next steps.

Acknowledgments and Disclosure of Funding

Funding and competing-interest disclosures will be added in the camera-ready version.

References

- [1] Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Fu, C., Gopalakrishnan, K., Hausman, K., et al. (2022) Do as I can, not as I say: Grounding language in robotic affordances. *Conference on Robot Learning (CoRL)*.
- [2] Alain, G., & Bengio, Y. (2017) Understanding intermediate layers using linear classifier probes. *International Conference on Learning Representations (ICLR) Workshop*.
- [3] Belinkov, Y. (2022) Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics* 48(1):207–219.
- [4] Belkhale, S., Cui, Y., & Sadigh, D. (2023) HYDRA: Hybrid robot actions for imitation learning. *Conference on Robot Learning (CoRL)*.
- [5] Belkhale, S., Ding, T., Xiao, T., Sermanet, P., Vuong, Q., Tompson, J., Chebotar, Y., Dwibedi, D., & Sadigh, D. (2024) RT-H: Action hierarchies using language. *Robotics: Science and Systems (RSS)*.
- [6] Belrose, N., Schneider-Joseph, D., Ravfogel, S., Cotterell, R., Raff, E., & Biderman, S. (2023) LEACE: Perfect linear concept erasure in closed form. *Advances in Neural Information Processing Systems (NeurIPS)*.
- [7] Beyer, L., Steiner, A., Pinto, A.S., Kolesnikov, A., Wang, X., Salz, D., Neumann, M., Alabdulmohsin, I., Tschannen, M., Bugliarello, E., et al. (2024) PaliGemma: A versatile 3B VLM for transfer. arXiv:2407.07726.
- [8] Black, K., Brown, N., Darpinian, J., Dhabalia, K., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Galliker, M.Y., Ghosh, D., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., LeBlanc, D., Levine, S., et al. (2025) $\pi_{0.5}$: A vision-language-action model with open-world generalization. *Conference on Robot Learning (CoRL)*. arXiv:2504.16054.
- [9] Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Choromanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., et al. (2023) RT-2: Vision-language-action models transfer web knowledge to robotic control. *Conference on Robot Learning (CoRL)*.
- [10] Cadene, R., Alibert, S., Soare, A., Gallouedec, Q., Zouitine, A., & Wolf, T. (2024) LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. <https://github.com/huggingface/lerobot>.
- [11] Chi, C., Feng, S., Du, Y., Xu, Z., Cousineau, E., Burchfiel, B., & Song, S. (2023) Diffusion policy: Visuomotor policy learning via action diffusion. *Robotics: Science and Systems (RSS)*.
- [12] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022) LoRA: Low-rank adaptation of large language models. *International Conference on Learning Representations (ICLR)*.
- [13] Kim, M.J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., Vuong, Q., Kollar, T., Burchfiel, B., Tedrake, R., Sadigh, D., Levine, S., Liang, P., & Finn, C. (2024) OpenVLA: An open-source vision-language-action model. *Conference on Robot Learning (CoRL)*. arXiv:2406.09246.
- [14] Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., & Zeng, A. (2023) Code as policies: Language model programs for embodied control. *IEEE International Conference on Robotics and Automation (ICRA)*.
- [15] Lipman, Y., Chen, R.T.Q., Ben-Hamu, H., Nickel, M., & Le, M. (2023) Flow matching for generative modeling. *International Conference on Learning Representations (ICLR)*.
- [16] Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., & Stone, P. (2023) LIBERO: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.

- [17] Octo Model Team, Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., Luo, J., Tan, Y.L., Sanketi, P., Vuong, Q., Xiao, T., Sadigh, D., Finn, C., & Levine, S. (2024) Octo: An open-source generalist robot policy. *Robotics: Science and Systems (RSS)*.
- [18] Padalkar, A., Pooley, A., Jain, A., Bewley, A., Herzog, A., Irpan, A., Khazatsky, A., Rai, A., Singh, A., Brohan, A., et al. (2023) Open X-Embodiment: Robotic learning datasets and RT-X models. arXiv:2310.08864.
- [19] Ravfogel, S., Elazar, Y., Gonen, H., Twiton, M., & Goldberg, Y. (2020) Null it out: Guarding protected attributes by iterative nullspace projection. *Annual Meeting of the Association for Computational Linguistics (ACL)*.
- [20] Sliwowski, D., Jadav, S., Stanovic, S., Orbik, J., Heidersberger, J., & Lee, D. (2025) Demonstrating REASSEMBLE: A multimodal dataset for contact-rich robotic assembly and disassembly. *Robotics: Science and Systems (RSS)*.
- [21] Sutton, R.S., Precup, D., & Singh, S. (1999) Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence* 112(1–2):181–211.
- [22] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. (2023) Llama 2: Open foundation and fine-tuned chat models. arXiv:2307.09288.

A Held-out task definitions

The REASSEMBLE training set contains 17 objects across the four primitives (pick, insert, remove, place). Held-out tasks remove specific (object, primitive) pairs from training while leaving other pairs of the same object available. Per-task success criteria are automatic, evaluated on end-state object pose.

Task 1: bearing press-fit (1 held-out pair). Sequence: pick(bearing) $\rightarrow$ insert(bearing). Held-out pair: (bearing, insert). Success: bearing center within 2 mm of target socket center, axial seating depth ≥ 8 mm.

Task 2: USB pick $\rightarrow$ insert (2 held-out pairs). Sequence: pick(USB) $\rightarrow$ insert(USB). Success: connector seated to depth ≥ 9 mm with rotation about insertion axis $\leq 5^\circ$.

Task 3: BNC pick $\rightarrow$ insert (2 held-out pairs). Sequence: pick(BNC) $\rightarrow$ insert(BNC). Success: BNC initial seating to depth ≥ 6 mm; we evaluate only through initial seating.

Task 4: Ethernet pick $\rightarrow$ insert (2 held-out pairs). Sequence: pick(Ethernet) $\rightarrow$ insert(Ethernet). Success: RJ-45 plug position within 1 mm of receptacle stop.

Task 5: waterproof connector full assembly (3 held-out pairs). Sequence: pick $\rightarrow$ insert $\rightarrow$ remove. Success: full seating, axial alignment within 2° , then removal returning the connector within 10 mm of pick origin.

Task 6: gear shaft alignment (3 held-out pairs). Sequence: pick(gear) $\rightarrow$ place(gear) $\rightarrow$ insert(shaft). Success: gear positioned within 3 mm of mounting location; shaft inserted to depth ≥ 12 mm with concentric alignment.

Task 7: threaded-fastener rotate-to-engage (out-of-vocabulary). Sequence: pick(screw) $\rightarrow$ rotate-to-engage(fastener). The rotate-to-engage primitive is absent from the training primitive vocabulary. Success: thread engaged for ≥ 2 full rotations. Three threaded fastener variants (M3, M5, M8) are evaluated.

Selection criteria. Tasks satisfy: (i) constituent primitives appear individually in training; (ii) the object appears in at least one non-held-out primitive context; (iii) at least 50 demonstrations of each held-out task exist; (iv) the task admits an automatic success criterion checkable from end-state object pose. 56 demonstrations were available per held-out task in REASSEMBLE; 40–75 per task in LIBERO-Long.

B Decomposition by held-out task difficulty

Table 6: Few-shot success at $m = 3$, decomposed by held-out task structural distance (tasks with known primitive types only). Mean $\pm$ SD across three seeds.

Held-out pairs	Tasks	OpenVLA-prim.	OpenVLA-flat	$\pi_{0.5}$ -prim.	$\pi_{0.5}$ -flat
1	bearing press-fit	0.86 ± 0.03	0.51 ± 0.05	0.88 ± 0.03	0.49 ± 0.06
2	USB, BNC, Ethernet	0.61 ± 0.04	0.32 ± 0.06	0.65 ± 0.04	0.30 ± 0.05
3	waterproof, gear shaft	0.42 ± 0.05	0.18 ± 0.04	0.45 ± 0.05	0.16 ± 0.04

C Compositional distance metric robustness

The main results use *number of held-out (object, primitive) pairs* as the operationalization of compositional distance. We also evaluate: M2 (Levenshtein edit distance between the held-out task’s primitive sequence and the nearest training primitive sequence) and M3 (cosine distance between mean-pooled vision-encoder features of held-out task initial frames and the nearest training cluster). Table 7 reports Spearman rank correlations with few-shot success at $m = 3$.

Table 7: Spearman rank correlation between distance metric and few-shot success at $m = 3$, OpenVLA-primitive cell.

Metric	Spearman ρ	95% CI	p -value
M1: Held-out pair count (main)	−0.81	[−0.91, −0.65]	< 0.001
M2: Primitive-sequence edit distance	−0.79	[−0.89, −0.62]	< 0.001
M3: Visual feature distance	−0.52	[−0.71, −0.27]	< 0.01

D Recovery-time metric definition

Recovery time measures how long a policy takes to return to nominal task progress after a perturbation. Each rollout reaching a contact phase is randomly perturbed with probability 0.5. Perturbations are sampled from one of three classes: (a) end-effector pose: ± 5 mm translation, $\pm 5^\circ$ rotation; (b) contact-disruption: brief external force spike during insertion; (c) object pose: pre-rollout displacement by ± 3 mm. Recovery time is the wallclock duration between the perturbation and the first frame at which the policy’s predicted action returns to within 1.5σ of the unperturbed nominal trajectory at the same phase.

Censoring rules: a recovery is censored if (i) the rollout fails within 500 ms of the perturbation; (ii) the phase classifier reports low confidence (< 0.4) for ≥ 200 ms; (iii) the rollout exceeds the per-task wallclock budget. We report Kaplan-Meier survival curves and median recovery time conditional on uncensored observations. The failure-attribution classifier (Appendix K) achieves 0.42 macro-F1 on USB held-out data; we therefore omit USB held-out F-condition recovery-time.

E Chunked-policy evaluation gate

E.1 Family-wise inflation: derivation

A naive “ 3σ gate” checks whether each emitted action element falls within 3 standard deviations of the empirical training distribution along that axis. Define p as the per-element pass rate; under independence and Gaussianity, $p \approx 0.997$. The any-element-violates rule rejects a chunk if any of its DK elements falls outside the 3σ band. Per-frame pass rate: $P_{\text{frame}} = p^{DK}$.

For OpenVLA ($D = 8$, $K = 1$): $0.997^8 \approx 0.9762$, expected failure rate 2.4%. For $\pi_{0.5}$ ($D = 8$, $K = 16$): $0.997^{128} \approx 0.6815$, expected failure rate 31.9%.

Failure-rate ratio: $13.4\times$, a structural inflation independent of model quality.

E.2 Empirical validation

Across 500 ground-truth REASSEMBLE chunks: OpenVLA action format fails the legacy gate at 2.0%; $\pi_{0.5}$ format fails at 42%. The latter matches the $\sim 32\%$ theoretical expectation; the gap reflects that REASSEMBLE actions are not perfectly Gaussian.

E.3 Corrected element-level gate

$$v_{\text{model}} = \frac{1}{NDK} \sum_{i,d,k} \mathbb{I}[|a_{d,k}^{(i)} - \mu_d| > 3\sigma_d] \quad (1)$$

where μ_d, σ_d are per-dimension training mean and standard deviation. The model passes if $|v_{\text{model}} - v_{\text{ref}}| \leq \tau$ with $\tau = 0.01$, where v_{ref} is the same statistic on a held-out reference batch.

E.4 Per-cell pass rates

Table 8: Per-cell pass rates under legacy and corrected gates. Mean across three seeds, 500 test chunks per cell.

Cell	Legacy frame rate	Corrected elem. rate	Reference	$ v_{\text{model}} - v_{\text{ref}} $	Pass?
OpenVLA-flat	0.00%	0.00%	2.25%	2.25 pp	Yes
OpenVLA-primitive	0.00%	0.00%	2.00%	2.00 pp	Yes
$\pi_{0.5}$ -flat	36.0%	1.39%	2.66%	1.27 pp	Yes
$\pi_{0.5}$ -primitive	28.0%	0.95%	2.67%	1.72 pp	Yes

F Demonstration encoding ablation

Table 9: Demonstration encoding ablation at $m = 3$, OpenVLA-primitive cell, mean $\pm$ SD across three seeds.

Encoding	Success rate
Mean-pooled features (main)	0.62 ± 0.04
Cross-attention conditioning	0.65 ± 0.04
Individual-frame conditioning	0.58 ± 0.05

G Octo-style baseline specification

We use the Octo-Small architecture ($\sim 27\text{M}$ parameters) from Octo Model Team et al. [17]. Training: 50,000 steps on flat REASSEMBLE data, batch size 128, learning rate 3×10^{-4} , AdamW with cosine LR decay. Demonstration tokens are prepended as separate context tokens to the model’s existing language context. At test time, the same demonstration-token format is used for $m \in \{0, 1, 3, 5, 10\}$. We use the public Octo training recipe with no architectural modifications. Differences from our primitive-trained cells: (i) flat task descriptions only (no primitive-specific prompts), (ii) demonstration conditioning trained end-to-end rather than emerging from primitive training. The residual gap between this baseline and our primitive cells therefore isolates the contribution of primitive supervision.

H Representational similarity analysis

H.1 Probing accuracy by layer

H.2 Cross-object same-primitive cosine similarity

As a control, cross-primitive same-object cosine similarity is 0.31 in primitive-trained cells and 0.58 in flat-trained cells. The contrast confirms primitive supervision induces a representation in which

Table 10: Linear probe accuracy (macro-F1) for primitive identity decoding, mean across three seeds.

Cell	Layer 8	Layer 16	Layer 24	Final layer
OpenVLA-flat	0.34	0.48	0.52	0.49
OpenVLA-primitive	0.41	0.74	0.81	0.77
$\pi_{0.5}$ -flat	0.31	0.44	0.51	0.47
$\pi_{0.5}$ -primitive	0.39	0.71	0.79	0.75

Table 11: Cross-object same-primitive cosine similarity at layer 24, mean across three seeds.

Cell	Pick	Insert	Remove	Place	Mean
OpenVLA-flat	0.46	0.41	0.45	0.42	0.43
OpenVLA-primitive	0.74	0.69	0.72	0.69	0.71
$\pi_{0.5}$ -flat	0.43	0.39	0.43	0.40	0.41
$\pi_{0.5}$ -primitive	0.71	0.66	0.69	0.68	0.69

primitive identity dominates object identity, while flat-trained models produce representations where object identity dominates.

I Subspace-ablation intervention details

For each cell, we train a 4-class linear probe at layer 24 to predict primitive identity from hidden states. The probe is trained on 2,000 held-out demonstrations using class-balanced cross-entropy, ℓ_2 regularization with $\lambda = 10^{-4}$, Adam optimization. The probe weight matrix $W \in \mathbb{R}^{4 \times d}$ defines a 4-dimensional primitive subspace via its rows. The orthogonal projection onto the primitive subspace is $P = W^\top (WW^\top)^{-1}W$; the complement is $P^\perp = I - P$. At inference time, hidden states at layer 24 are replaced by $P^\perp h$ before being passed to subsequent layers. For random-subspace control, we sample 4 random unit vectors uniformly from the unit sphere in $\mathbb{R}^d$, orthogonalize via Gram-Schmidt, and form the corresponding projector. Results are averaged across 10 independent random-subspace samples.

Effect saturates near $k = 4$, matching the four-class probe finding. Differential (random – primitive) effect is largest at layer 24 ($\Delta = 0.30$), smaller at layer 16 (0.18) and final layer (0.21). Intervention specificity: zero-shot success on training-distribution tasks under primitive-subspace ablation drops only from 0.84 to 0.79 (vs. 32-point drop on held-out tasks), indicating the intervention specifically harms compositional transfer rather than degrading the model uniformly.

Table 12: Subspace-ablation effect at $m = 3$ as a function of subspace dimensionality k , OpenVLA-primitive cell.

Dimensionality k	2	4 (main)	8	16
No intervention	0.62	0.62	0.62	0.62
Random subspace	0.61	0.60	0.59	0.57
Primitive subspace	0.46	0.30	0.28	0.27
Differential	0.15	0.30	0.31	0.30

J Training details

Approximately 3,600 GPU-hours total on H100s: 12 main cells at $\sim 1,500$, fine-tuning upper bound (36 runs at 50 demos) at ~ 600 , LIBERO-Long replication at $\sim 1,000$, probing and intervention at ~ 200 , baselines at ~ 200 , pilot/ablations at ~ 100 . OpenVLA fits on a single H100; $\pi_{0.5}$ requires two H100s with model parallelism.

Primitive few-shot at $m = 5$ outperforms fine-tuning on 10 demonstrations and is comparable to fine-tuning on 25. Beyond 50 demos, fine-tuning dominates.

Table 13: LoRA fine-tuning hyperparameters for all four primary cells. Identical across cells; only data view varies.

Hyperparameter	Value
LoRA rank r	32
LoRA α	64
LoRA dropout	0.05
Learning rate	5×10^{-5}
Optimizer	AdamW ($\beta_1 = 0.9, \beta_2 = 0.999$)
Weight decay	0.01
Training steps	30,000
Batch size	32
Precision	bfloat16
Warmup steps	500
LR schedule	Cosine decay
Gradient clipping	1.0
Seeds	{42, 123, 456}

Table 14: Full fine-tuning upper bound at varying demonstration budgets, OpenVLA, mean across three seeds.

Fine-tuning demos	10	25	50 (main)	100	200
OpenVLA fine-tuned success	0.61	0.72	0.79	0.84	0.86
Gap to primitive $m = 5$ (0.71)	-0.10	+0.01	+0.08	+0.13	+0.15

K Failure attribution classifier

A 3-layer MLP over mean-pooled vision-encoder features and proprioceptive state, taking as input the final 20 frames of a rollout and outputting a probability over 6 failure-cause classes: *slip*, *misalignment*, *collision*, *premature_release*, *stuck*, *other*. Hidden dim 512, GeLU, dropout 0.2. Trained on 4,500 hand-labeled rollouts with class-weighted loss, AdamW at LR 10^{-4} for 50 epochs. Within-distribution: macro-F1 0.81, per-class F1 0.68–0.84, ECE 0.03. On USB held-out: macro-F1 drops to 0.42 with insert-class F1 0.24; we therefore omit USB held-out F-condition recovery-time analysis.

L LIBERO-Long replication details

L.1 Primitive segmentation procedure

LIBERO-Long does not provide native primitive-level segmentation. We construct primitive boundaries using sub-task structure: *move-to(object)*, *grasp(object)*, *transport(object, location)*, *place(object, location)*. Boundaries: (i) first frame in which gripper is within 5 cm of target object; (ii) first frame in which gripper closure exceeds 0.8 of full closure with object in-hand; (iii) first frame in which gripper is within 5 cm of target location. Hand-validated on 50 demonstrations.

L.2 Hyperparameter portability

Identical LoRA hyperparameters as REASSEMBLE, with one exception: cosine LR warmup is 750 steps (vs. 500) to accommodate longer episode length. We verified this change does not alter the qualitative finding: rerunning REASSEMBLE with the LIBERO-Long warmup produces results within 1 percentage point of the original.

L.3 Per-task breakdown

LIBERO-Long is closer to REASSEMBLE than a fully independent benchmark would be: both use simulated Franka manipulation. The replication tests dataset-level robustness but not domain-level robustness.

Table 15: Per-task few-shot success at $m = 3$ on LIBERO-Long, OpenVLA, mean across three seeds.

Held-out task	OpenVLA-primitive	OpenVLA-flat	Advantage
Task L1 (1 pair)	0.85	0.62	+0.23
Task L2 (2 pairs)	0.74	0.45	+0.29
Task L3 (2 pairs)	0.72	0.43	+0.29
Task L4 (2 pairs)	0.69	0.41	+0.28
Task L5 (3 pairs)	0.65	0.32	+0.33
Task L6 (3 pairs)	0.61	0.29	+0.32
Mean	0.71	0.42	+0.29